

LeetCode 457: Circular Array Loop

Notes & Intuition

Companies: Goldman Sachs, Nutanix

1. Problem Understanding

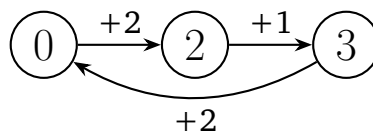
Every index in the given array `nums` acts as a **node** in a graph. The value at the index determines the jump length and direction.

Example Trace

Let `nums` = [2, -1, 1, 2, 2].

- Start at index 0: Value is 2. Next index = $(0 + 2) \pmod{5} = 2$.
- At index 2: Value is 1. Next index = $(2 + 1) \pmod{5} = 3$.
- At index 3: Value is 2. Next index = $(3 + 2) \pmod{5} = 0$.

Since the graph representation is not explicitly given, the underlying graph dictates a cycle leading to:



Crucial Observations (Strict Checks)

1. **No Mixed Directions:** A cycle cannot mix positive and negative directions.

- *Example:* [1, -1, 1, -1]. Index 0 → 1 and 1 → 0 looks like a cycle, but it involves moving right (positive) then left (negative). **Not Allowed.**

2. **Length 1 Cycle is Invalid:** A cycle of length 1 is not permitted.

- *Example:* `nums` = [3, 1, 2].
At $i = 0$, `nums`[0] = 3. Next index = $(0 + 3) \pmod{3} = 0$.
This forms a self-loop (length 1 cycle). **Not Allowed.**

2. Intuition & Approach (Floyd's Cycle Finding)

We utilize **Floyd's Tortoise and Hare** algorithm, tracking indices efficiently without extra space.

Step-by-Step Strategy

1. **Fix Direction:** Ensure that throughout the traversal, the direction (sign of the element) remains consistent with the starting node.
2. **Pointer Movement:**
 - slow pointer moves +1 step.
 - fast pointer moves +2 steps.
3. **Meet Condition:** If slow and fast meet, we confirm a cycle exists. However, we must ensure the cycle length $L > 1$. If valid, return true.
4. **Marking Visited Paths:** To avoid reprocessing nodes and traversing identical dead-ends, mark all processed nodes in a path as 0. Since 0 implies "neither forward nor backward," we can reuse the nums array itself without $O(N)$ extra space.

Next Index Calculation

To properly fetch the next index considering cyclic bounds and negative values:

$$\text{next_index} = \left((i + \text{nums}[i]) \pmod n + n \right) \pmod n$$

3. C++ Implementation

```
1 class Solution {
2 public:
3     int n;
4
5     int nextidx(int startpos, vector<int>& nums, int dir) {
6         int currrdir = nums[startpos] > 0;
7         if(currrdir != dir) return -1; // Direction mismatch
8
9         // Safe modulo logic for both positive and negative jumps
10        int next = (((startpos + nums[startpos]) % n) + n) % n;
11
12        if(next == startpos) return -1; // Length 1 cycle is invalid
13        return next;
14    }
15
16    bool circularArrayLoop(vector<int>& nums) {
17        n = nums.size();
18
19        for(int i = 0; i < n; i++) {
20            if(nums[i] == 0) {
21                continue; // Already processed
22            }
23
24            int slow = i;
25            int fast = i;
26
27            // Check initial direction to ensure cycle consistency
28            int dir;
29            if(nums[i] > 0) dir = 1;
30            else dir = 0;
31
32            // Floyd's Cycle Detection
33            while(true) {
34                slow = nextidx(slow, nums, dir);
35                if(slow == -1) break;
36
37                fast = nextidx(fast, nums, dir);
38                if(fast == -1) break;
39
40                fast = nextidx(fast, nums, dir);
41                if(fast == -1) break;
42
43                if(slow == fast) return true; // Cycle found
44            }
45
46            // If cycle not found, mark the entire path as 0 (visited)
47            int curr = i;
48            while(true) {
49                int currrdir = nums[curr] > 0;
50
51                if(nums[curr] == 0) break; // Already marked
52                if(currrdir != dir) break; // Direction mismatch
53
54                int next = (((curr + nums[curr]) % n) + n) % n;
55            }
```

```
56         if(next == curr) break; // Reached a 1-length cycle
57
58         nums[curr] = 0; // Mark visited
59         curr = next;
60     }
61 }
62
63 return false;
64 }
65 };
```